

C++&Qt Day04 hw1 보고서

작성자: Doyeon Park

작성일: 2026.09.08

목차

1. 과제 개요
2. UI 구성
3. UDP 통신 구현
4. 메시지 전송
5. 전체 코드 동작 과정
6. 실행 결과

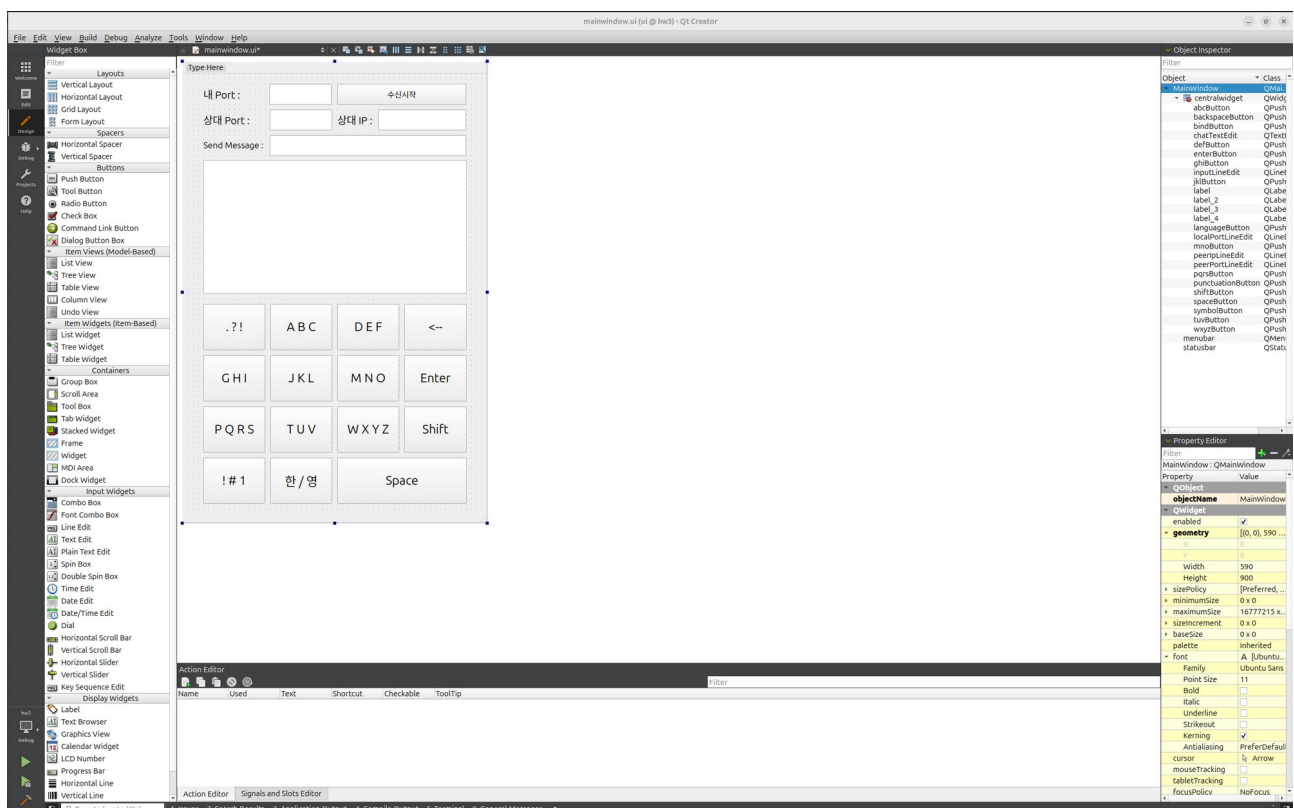
1. 과제 개요

이번 과제에서는 Qt 의 UDP 통신 기능을 이용하여 두 사용자가 서로 문자 메시지를 주고 받을수 있도록 프로그래밍 하였다.

카톡과 같은 느낌으로 구현하기 위해 이전 과제에서 구현했던 천지인 키보드를 재사용하였다. 천지인에서 한글, 영어, 숫자 및 특수문자를 입력하고 Enter 버튼을 누르면 입력한 문장이 UDP 를 통해 상대방에게 전송되도록 구현하였다.

2. UI 구성

프로그램의 UI 구성은 다음과 같다.

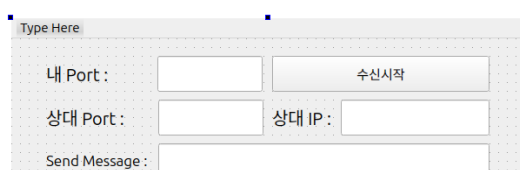


프로그램의 화면은 크게 UDP 설정 영역, 채팅 영역, 메시지 입력 영역, 천지인 키보드 영역으로 구성하였다.

2.1 UDP 설정 영역

UDP 통신을 위해 내 Port, 상대 Port, 상대 IP, 수신시작버튼으로 구성하였다.

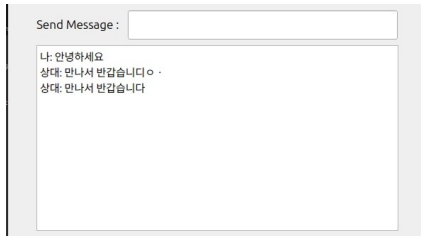
내 Port 는 현재 프로그램이 UDP 데이터를 받을 포트 번호이고, 상대 Port 는 메시지를 전송할 상대 프로그램의 포트 번호이다. 상대 IP 에는 메시지를 전송할 상대 컴퓨터의 IP 주소를 입력하고 수신시작 버튼을 누르면 입력한 내 Port 에 bind()하여 UDP 데이터를 받을 준비를 한다.



2.2 채팅창

채팅 내용은 QTextEdit 을 이용하여 표시하였다.

채팅 기록을 사용자가 직접 수정할 필요가 없기 때문에 readOnly 상태로 설정하였다.

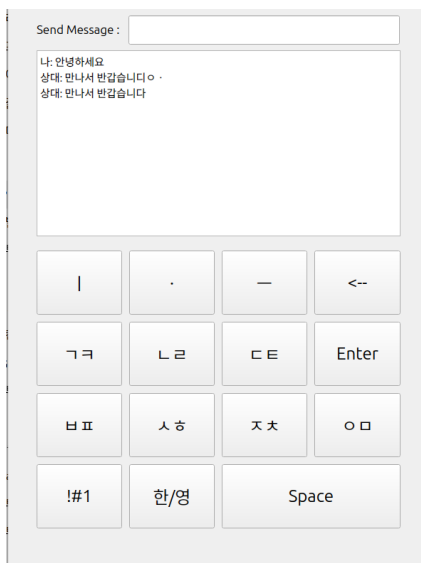


2.3 천지인 키보드

메시지 입력 부분은 이전 과제에서 제작한 천지인 키보드를 재사용 하였다.

기존 천지인 키보드에서는 한글, 영어 대소문자, 숫자 및 특수문자를 입력할 수 있었으며, 버튼을 여러 번 누르면 같은 버튼에 포함된 문자가 순서대로 변경되도록 구현되어 있었다.

이번 과제에서는 기존 입력 기능을 유지하였고 Enter 버튼만 파일 저장에서 UDP 메시지 전송기능으로 변경하였다.



3. UDP 통신 구현

3.1 QUdpSocket

UDP 통신을 위해 Qt Network 에서 제공하는 QudpSocket 클래스를 사용하였다.

헤더 파일에 다음과 같이 UDP 소켓을 선언하였다.

```
QudpSocket *udpSocket;
```

이후 프로그램 생성자에서 실제 객체를 생성하였다.

```
UdpSocket = new QudpSocket(this);
```

이 객체를 이용하여 UDP 데이터 송신과 수신을 모두 처리하였다.

3.2 UDP 수신 준비

UDP 데이터를 받기 위해 사용자가 수신 시작 버튼을 누르면 입력한 Port 번호를 가져온 후 bind()를 실행하도록 하였다.

```
UdpSocket -> bind(  
    QhostAddress::AnyIPv4,  
    localPort  
);
```

bind()를 사용하면 현재 프로그램이 지정한 Port 로 돌아오는 UDP 데이터를 받을 수 있다.

예를 들어 내 Port 를 10004 로 설정하면 이 프로그램은 10004 번 Port 로 들어오는 데이터를 기다리게 된다. 실제 구현에서도 입력된 localPort 값을 이용해 QhostAddress::AnyIPv4 에 bind 하도록 작성하였다.

3.3 Signal / Slot 을 이용한 데이터 수신

UDP 데이터가 도착했는지를 계속 반복문으로 검사하지 않고 Qt 의 Signal/Slot 기능을 이용하였다.

다음은 관련 코드이다.

```
Connect(  
    udpSocket,  
    &QudpSocket::readyRead,  
    this,  
    &MainWindow::udpRead  
);
```

readyRead 는 UDP 데이터가 도착하면 발생하는 Signal 로 도착 순서는 다음과 같다.

UDP 데이터 도착 → readyRead Signal → udpRead() 실행 → 데이터 읽기

3.4 UDP 데이터 수신

udpRead() 에서는 아직 읽지 않은 데이터그램이 있는동안 데이터를 읽는다.

```
While (udpSocket → hasPendingDatagrams())  
{  
    QNetworkDatagram datagram =  
        udpSocket -> receiveDatagram();  
  
    QByteArray data =  
        datagram.data();  
  
    QString message =  
        QString::fromUtf8(data);  
}
```

receiveDatagram()을 이용하여 데이터그램을 받고, 내부 데이터를 QByteArray 형태로 가져왔다.

그 후에 QString::fromUtf8()을 이용하여 화면에 출력할 수 있는 문자열로 변환하였다.

4. 메시지 전송

천지인 키보드에서 문장을 작성한 후 Enter 버튼을누르면 메시지를 전송한다.

먼저 아직 확정되지 않은 문자를 확정한다.

```
CommitCurrentCharacter();

if (koreanMode)
{
    commitKoreanSyllable();
}
```

천지인 키보드에서 마지막으로 입력하고 있는 글자가 아직 조합 중일 수도 있기 때문에 이 과정을 거친다.

그 후 상대 IP 와 상대 Port 값을 가져온다.

```
QString peerIp =
    ui -> peerIpLineEdit
        -> text()
        .trimmed();

quint16 peerPort =
    ui -> peerPortLineEdit
        -> text()
        .toUShort();
```

메시지는 프로그램 내부에서는 QString 형식으로 관리하고 있기 때문에 UDP 로 전송하기 전에 QByteArray 로 변환하였다.

```
QByteArray data =
    committedText.toUtf8();
```

실제 전송은 다음과 같다.

```
UdpSocket -> writeDatagram(
    data,
    peerAddress,
    peerPort
);
```

즉 데이터가 전송되는 순서는 다음과 같다.

천지인 입력 → QString → toUtf8() → QByteArray → writeDatagram() → 상대 IP : 상대 Port

5. 전체 코드 동작 과정

프로그램 실행 → 내 Port 입력 → 상대 IP / Port 입력 → 수신 시작 → 천지인 키보드로 문자 입력 → Enter
→ 현재 입력 문자 확정 → UTF-8 QbyteArray 변환 → UDP 데이터그램 전송 → 상대 프로그램 readyRead 발생
→ udpRead() → receiveDatagram() → 채팅창에 메시지 출력

6. 실행 결과

두 대의 컴퓨터에서 프로그램을 실행하고 각각 자신의 Port 를 열어둔 후 상대방의 IP 와 Port 를 입력하였다.

한 사용자가 천지인 키보드로 문장을 작성하고 Enter 버튼을 누르면 자신의 화면에는

나: (메시지)

가 출력되고, 상대 프로그램에는

상대: (메시지)

가 출력되었다.

한글, 영어 등의 문자도 UTF-8 방식으로 변환하여 전송함으로써 메시지를 주고받을 수 있었다.

